

Portfolio App - SD2 Lab Project Report

Introduction

This report details the development of my personal portfolio application using Flutter, created as a culminating project for the SD2 Lab course. The project demonstrates the practical application of modern cross-platform development concepts, reactive state management and interactive user interface design.

This project reflects the core teachings of the SD2 lab by focusing on code architecture, maintainability, and providing immediate visual feedback to user interactions.

1. GetX Architecture Integration

To manage the state and logic of the application independently from the UI layer, the **GetX architecture** was heavily utilized. This reflects our lab lessons on separating business logic from view elements.

- **State Management:** A `PortfolioController` extending `GetxController` is used to handle dynamic states. The application theme (`isDarkTheme`) and the list of projects (`projects`) are marked as observable variables (`.obs`).
- **Reactive UI:** The `Obx` widget is utilized to make specific parts of the UI reactive. For instance, `GetMaterialApp` is wrapped in an `Obx` widget, meaning the entire application's theme switches instantly when `controller.toggleTheme()` is called, without

needing to rebuild the entire widget tree using `setState`. The `ListView.builder` for the projects is also reactive.

- **Dependency Injection:** `Get.put(PortfolioController())` is used in `main.dart` to initialize the controller into memory, allowing any widget (like the `AnimatedThemeButton` or `HomeView`) to access the controller instantly using `Get.find()`.
-

2. Dynamic UI Animations

A strict requirement of the SD2 lab was to implement size and color transition animations on at least 3 different UI components. To achieve a modern, interactive aesthetic, the following 4 components were built using implicit animations (`AnimatedContainer`, `AnimatedScale`, `AnimatedSize`):

1. Expandable Project Card (`ProjectCard`)

- **Interaction:** Clicking the project card acts as an accordion.
- **Animation:** It uses an `AnimatedContainer` and `AnimatedSize` to physically expand and reveal the project description. Simultaneously, the background color transitions to a highlighted purple tint, and a deep purple border fades in.

2. Interactive Skill Badges (`AnimatedSkillBadge`)

- **Interaction:** Pressing a skill badge.
- **Animation:** Utilizes `AnimatedScale` and `AnimatedContainer`. When tapped, the badge scales down slightly (simulating a physical press) and its background color shifts from deep purple to a lighter accent color.

3. Profile Image Card (`AnimatedProfileCard`)

- **Interaction:** Tapping and holding the profile picture.
- **Animation:** Scales the entire profile picture container and transitions the container's shadow and background tint colors to provide tactile feedback.

4. Theme Toggle Button (`AnimatedThemeButton`)

- **Interaction:** Clicking the Day/Night switch.
- **Animation:** Changes its dimensions, morphs its

background color (representing day/night), and dynamically switches the icon with a scaling effect using `AnimatedSwitcher`.

3. Real-world Integrations

Reflecting practical application development taught in the lab, the project integrates real-world tools:

- **URL Launcher:** The `url_launcher` package is used to execute native OS intents, allowing the GitHub, LinkedIn, and Project repository buttons to successfully open the user's web browser.
- **Responsive Theming:** The app rigorously uses `Theme.of(context).colorScheme` to ensure that custom containers, text, and outlined buttons adapt flawlessly to both Light and Dark modes.

Conclusion

This project perfectly encapsulates the objectives of the SD2 Lab course. By moving beyond static layouts and incorporating reactive state management (GetX) alongside dynamic, gesture-driven animations, the application demonstrates a deep understanding of the Flutter lifecycle. The clean separation of concerns (Models, Views, Controllers, Widgets) provides a solid architectural foundation for building more complex applications in the future.